UNIVERSIDAD POLITÉCNICA
BICENTENARIO

Práctica 4: Control de intensidad de un foco con dimmer y servomotor.

Presenta:

Rubén Antonio Arredondo Villegas

Edgar ivan Gutierrez Gutierrez

Roberto Leonel Mendoza Nuñez

Ailton Alejandro Trujillo López

Matricula:

24033101

24032957

24033370

24033482

Asignatura:

Sistemas embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. a 15 de octubre del 2026

Contenido

Introducción 3
Marco Teórico 3
Desarrollo 4
Resultados 10
Observaciones y Conclusiones 14
Referencias 15

Lista de Figuras

Ilustración 1. Diagrama de conexiones del dimmer con servomotor 5
Ilustración 2. Pantalla LCD mostrando una intensidad de 11 % con el servomotor a 21° ... 11
Ilustración 3. Circuito ensamblado con el foco a baja intensidad 12
Ilustración 4. Foco encendido con una intensidad de 60 % (servomotor a 108°) 13
Ilustración 5. Vista general del circuito en funcionamiento 14

Introducción

El control de la intensidad luminosa de un foco es una aplicación común en la automatización de espacios, ya que permite ajustar la iluminación según las necesidades del usuario. Una forma sencilla de lograrlo en corriente alterna es mediante un dimmer, un circuito que regula la potencia que recibe la carga.

En esta práctica se controló la intensidad de un foco incandescente conectado a 120 V utilizando un módulo dimmer AC de 2000 W, cuya perilla es girada por un servomotor MG995. El servomotor es manejado por una ESP32 mediante dos botones, uno para subir y otro para bajar la intensidad, y una pantalla LCD 16x2 muestra el porcentaje de intensidad y la posición del servo en grados.

Marco Teórico

El ESP32 es un microcontrolador que integra entradas y salidas digitales, comunicación inalámbrica y múltiples periféricos que permiten desarrollar sistemas de control y automatización [1]. En esta práctica se utilizan sus pines GPIO (*General Purpose Input/Output*) como entradas digitales para leer los botones, con sus resistencias pull-up internas, y como salida para generar la señal de control del servomotor [1].

Servomotor y señal PWM

Un servomotor es un actuador que puede posicionarse en un ángulo determinado. Su posición se controla con una señal PWM de 50 Hz: el ancho del pulso, normalmente entre 0.5 ms y 2.4 ms, indica el ángulo entre 0° y 180° [3]. El MG995 es un servo de engranes metálicos con el par suficiente para girar la perilla del dimmer. En la ESP32 la señal se genera con la librería ESP32Servo, ya que la librería Servo de Arduino no es compatible con esta tarjeta [4].

Dimmer de corriente alterna

El dimmer AC regula la potencia que recibe la carga mediante control de fase: un tiristor (SCR o TRIAC) se activa en cierto punto de cada semiciclo de la señal de 60 Hz, de modo que el foco solo recibe una parte de la onda. El potenciómetro del módulo ajusta ese punto de disparo; mientras antes se activa el tiristor, mayor es el voltaje eficaz en el foco y mayor su brillo. Este tipo de regulación funciona bien con cargas resistivas como los focos incandescentes. Es importante considerar que todo el circuito del dimmer, incluido el potenciómetro, está conectado a la red eléctrica.

Pantalla LCD 16x2 con módulo I2C

La pantalla LCD de 16 columnas por 2 filas permite mostrar información al usuario sin necesidad de una computadora. Con el módulo I2C basado en el PCF8574 solo requiere dos líneas de datos (SDA y SCL) además de la alimentación, lo que reduce el número de conexiones con el microcontrolador [2].

Material requerido

- 1 tarjeta ESP32 con su base de expansión.
- 1 servomotor MG995.
- 1 módulo dimmer AC de 2000 W.
- 1 foco incandescente con soquet.
- 1 cable con clavija para la toma de corriente.
- 1 pantalla LCD 16x2 con módulo I2C.
- 2 botones pulsadores.
- Protoboard.
- Cables Dupont.
- Cinta para fijar el acople del servo.
- Cable USB.
- Computadora con Arduino IDE.

Asignación de pines

Componente	Pin del componente	Pin del ESP32
Servomotor MG995	Señal (naranja)	GPIO 18
Servomotor MG995	V+ (rojo)	5V (VIN)
Servomotor MG995	GND (café)	GND
Botón SUBIR	Terminal 1 / terminal 2	GPIO 32 / GND
Botón BAJAR	Terminal 1 / terminal 2	GPIO 33 / GND
Pantalla LCD (I2C)	SDA / SCL	GPIO 21 / GPIO 22
Pantalla LCD (I2C)	VCC / GND	5V (VIN) / GND

Desarrollo

La práctica se realizó en dos partes: primero se hizo el control del servomotor utilizando únicamente el monitor serial del IDE de Arduino y, una vez comprobado su funcionamiento, se añadió la pantalla LCD con los indicadores.

Para que el servomotor pudiera girar la perilla del dimmer, primero se adaptó una de las estrellas del servo al potenciómetro del dimmer. Después, esa estrella se pegó a otra estrella colocada en el eje del servo, de forma que al moverse el servomotor también girara el potenciómetro. Finalmente, ambos elementos se fijaron con cinta para dar mayor estabilidad al acople y evitar que se separaran con el movimiento.

En cuanto a las conexiones, la entrada del dimmer se conectó a la toma de corriente de 120 V y el foco a su salida, de modo que la energía del foco pasa a través del módulo. El servomotor se alimentó directamente desde la salida de 5 V de la ESP32, que a su vez se energizó con el cable USB de la computadora, y su señal de control se conectó al GPIO 18. Los botones SUBIR y BAJAR se conectaron a los GPIO 32 y 33 utilizando las resistencias pull-up internas, por lo que no fue necesario agregar resistencias externas.

En la primera parte, el programa movía el servo 3° con cada pulsación y mostraba en el monitor serial el ángulo y el porcentaje de intensidad. Con esta prueba se verificó que el recorrido del servo coincidiera con el de la perilla. Además, si se mantiene presionado un botón el servo sigue moviéndose, y la última posición se guarda en la memoria de la ESP32 para que al reiniciar no dé un giro brusco.

Posteriormente se añadió la pantalla LCD, conectada a los pines SDA (GPIO 21) y SCL (GPIO 22). En la primera fila muestra el porcentaje de intensidad y en la segunda la posición del servomotor en grados; ambos valores se actualizan cada vez que se presiona un botón.

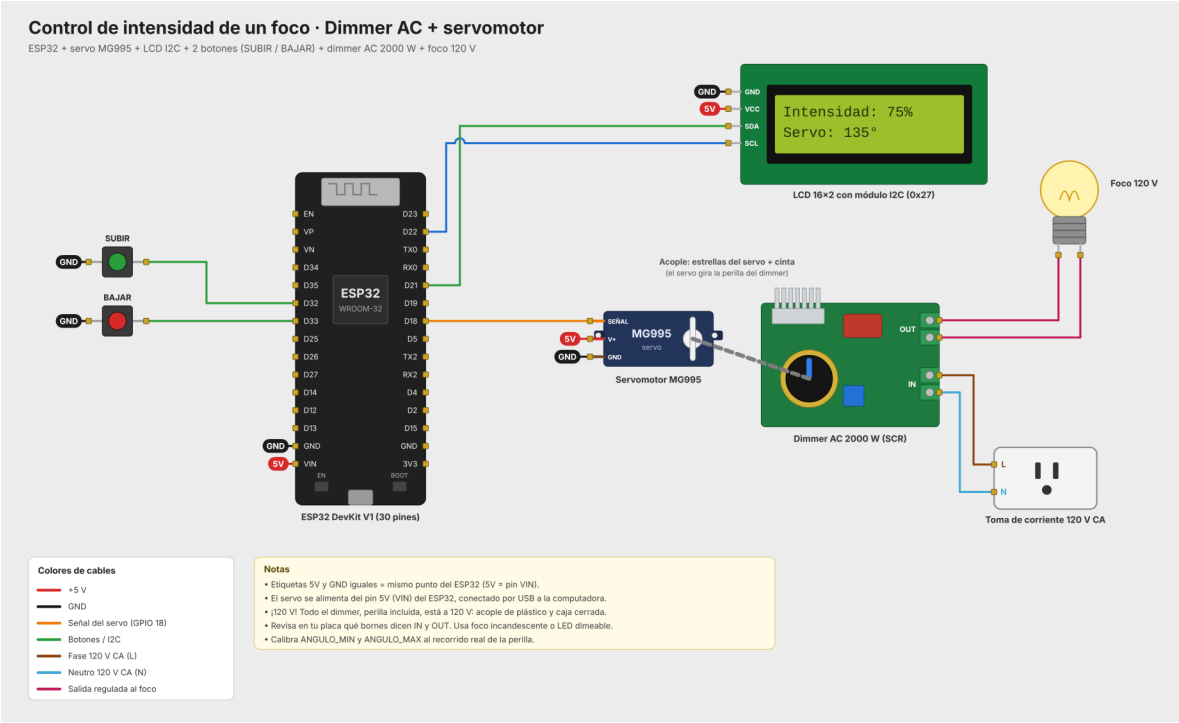


Ilustración 1. Diagrama de conexiones del dimmer con servomotor

En el diagrama se observa que el circuito requiere pocas conexiones: los dos botones, la señal del servomotor y las dos líneas de la pantalla LCD hacia la ESP32, mientras que la parte de 120 V se limita a la toma de corriente, el dimmer y el foco.

A continuación, se muestra el código empleado en el IDE de Arduino.

```
#include <ESP32Servo.h>
#include <Preferences.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---------------------- PINES ----------------------
#define PIN_SERVO      18
#define BTN_SUBIR      32
#define BTN_BAJAR      33
#define LCD_SDA        21
#define LCD_SCL        22

// ---------------------- AJUSTES ----------------------
const int  ANGULO_MIN       = 0;
const int  ANGULO_MAX       = 180;
const bool INVERTIR_SENTIDO = false;
const int  PASO_GRADOS      = 3;

// Pulsos del MG995 (en microsegundos) para 0 y 180 grados
const int PULSO_MIN_US = 500;
const int PULSO_MAX_US = 2400;

// Repeticion automatica al mantener presionado un boton
const unsigned long RETARDO_REPETICION_MS  = 400;  // espera antes de repetir
const unsigned long INTERVALO_REPETICION_MS = 60;  // velocidad de repeticion
const unsigned long DEBOUNCE_MS = 30;

// ---------------------- OBJETOS Y VARIABLES ----------------------
Servo servo;
Preferences prefs;
LiquidCrystal_I2C lcd(0x27, 16, 2);

const char GRADO = (char)223;  // simbolo de grados en la LCD

int angulo = ANGULO_MIN;   // angulo actual del servo
bool hayCambioSinGuardar = false;
unsigned long ultimoCambio = 0;
const unsigned long GUARDAR_DESPUES_MS = 2000;  // guarda 2 s despues del ultimo
cambio

struct Boton {
  int pin;
  bool presionado;
  unsigned long momentoPresion;
  unsigned long ultimaRepeticion;
  unsigned long ultimoCambioLectura;
  bool lecturaAnterior;
};

Boton botonSubir = { BTN_SUBIR, false, 0, 0, 0, HIGH };
Boton botonBajar = { BTN_BAJAR, false, 0, 0, 0, HIGH };
```

```
// ---------------------- DECLARACION DE FUNCIONES ----------------------
bool debeMover(Boton &b);
void moverIntensidad(int direccion);
void aplicarAngulo();
int  porcentajeIntensidad();
void actualizarLCD();
void imprimirLinea(int fila, const char* texto);

// ========================================================================
void setup() {
  Serial.begin(115200);


  pinMode(BTN_SUBIR, INPUT_PULLUP);
  pinMode(BTN_BAJAR, INPUT_PULLUP);


  // LCD: pequena espera para que arranque, e I2C a velocidad estandar
  delay(500);
  Wire.begin(LCD_SDA, LCD_SCL);
  Wire.setClock(100000);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  imprimirLinea(0, "Dimmer + servo");
  imprimirLinea(1, "Iniciando...");


  // Recuperar el ultimo angulo guardado (asi el servo no salta al encender)
  prefs.begin("dimmer", false);
  angulo = prefs.getInt("angulo", ANGULO_MIN);
  prefs.end();
  angulo = constrain(angulo, ANGULO_MIN, ANGULO_MAX);


  ESP32PWM::allocateTimer(0);
  servo.setPeriodHertz(50);  // los servos usan senal de 50 Hz
  servo.attach(PIN_SERVO, PULSO_MIN_US, PULSO_MAX_US);
  aplicarAngulo();


  delay(800);
  actualizarLCD();


  Serial.println("=== Dimmer con servomotor ===");
  Serial.print("Angulo inicial: "); Serial.print(angulo);
  Serial.print(" | Intensidad: ");  Serial.print(porcentajeIntensidad());
  Serial.println(" %");
}

// ========================================================================
void loop() {
  if (debeMover(botonSubir)) moverIntensidad(+1);
  if (debeMover(botonBajar)) moverIntensidad(-1);


  // Guardar la posicion en memoria cuando el usuario deja de mover
  // (no se guarda en cada paso para no desgastar la memoria flash)
  if (hayCambioSinGuardar && millis() - ultimoCambio > GUARDAR_DESPUES_MS) {
    prefs.begin("dimmer", false);
    prefs.putInt("angulo", angulo);
    prefs.end();
    hayCambioSinGuardar = false;
    Serial.println("Posicion guardada");
  }
}
```

```
// ============================================================================
// Devuelve true cuando el boton debe producir un paso:
//  - una vez al presionarlo
//  - y luego repetidamente mientras se mantiene presionado
bool debeMover(Boton &b) {
  bool lectura = digitalRead(b.pin);
  unsigned long ahora = millis();

  // Antirrebote: ignorar cambios muy rapidos
  if (lectura != b.lecturaAnterior) {
    b.ultimoCambioLectura = ahora;
    b.lecturaAnterior = lectura;
  }
  if (ahora - b.ultimoCambioLectura < DEBOUNCE_MS) return false;

  bool presionadoAhora = (lectura == LOW);

  if (presionadoAhora && !b.presionado) {        // se acaba de presionar
    b.presionado = true;
    b.momentoPresion = ahora;
    b.ultimaRepeticion = ahora;
    return true;
  }
  if (!presionadoAhora) {                        // se solto
    b.presionado = false;
    return false;
  }
  // Se mantiene presionado: repetir despues del retardo inicial
  if (ahora - b.momentoPresion > RETARDO_REPETICION_MS &&
      ahora - b.ultimaRepeticion > INTERVALO_REPETICION_MS) {
    b.ultimaRepeticion = ahora;
    return true;
  }
  return false;
}

// direccion: +1 = mas intensidad, -1 = menos intensidad
void moverIntensidad(int direccion) {
  if (INVERTIR_SENTIDO) direccion = -direccion;

  int nuevo = constrain(angulo + direccion * PASO_GRADOS, ANGULO_MIN, ANGULO_MAX);
  if (nuevo == angulo) return;  // ya esta en el limite

  angulo = nuevo;
  aplicarAngulo();
  actualizarLCD();
  hayCambioSinGuardar = true;
  ultimoCambio = millis();

  Serial.print("Angulo: ");        Serial.print(angulo);
  Serial.print(" | Intensidad: "); Serial.print(porcentajeIntensidad());
  Serial.println(" %");
}

void aplicarAngulo() {
  servo.write(angulo);
}

// % de intensidad segun la posicion dentro del recorrido calibrado
int porcentajeIntensidad() {
```

```
  int p = map(angulo, ANGULO_MIN, ANGULO_MAX, 0, 100);
  return INVERTIR_SENTIDO ? 100 - p : p;
}

// ========================================================================
// LCD 16x2
//   Fila 0: "Intensidad:  75%"
//   Fila 1: "Servo:     135° "
void actualizarLCD() {
  char linea[17];
  snprintf(linea, sizeof(linea), "Intensidad: %3d%%", porcentajeIntensidad());
  imprimirLinea(0, linea);
  snprintf(linea, sizeof(linea), "Servo:     %3d%c", angulo, GRADO);
  imprimirLinea(1, linea);
}

// Escribe un texto en la fila indicada y rellena con espacios hasta 16
// caracteres, para borrar restos de textos anteriores sin parpadeo
void imprimirLinea(int fila, const char* texto) {
  lcd.setCursor(0, fila);
  int n = 0;
  while (texto[n] != '\0' && n < 16) {
    lcd.print(texto[n]);
    n++;
  }
  while (n < 16) {
    lcd.print(' ');
    n++;
  }
}
```

Resultados

El código se cargó en la ESP32 sin errores y el servomotor respondió correctamente a los botones desde la primera prueba con el monitor serial. Al girar el servo, la perilla del dimmer se movía junto con él gracias al acople de las estrellas, y la intensidad del foco cambiaba de acuerdo con su posición.

Al agregar la pantalla LCD fue posible visualizar en todo momento el porcentaje de intensidad y los grados del servomotor. Con valores bajos, como 11 % (21°), el foco se mantenía prácticamente apagado, mientras que al aumentar la intensidad, por ejemplo a 60 % (108°), el foco encendía con un brillo notable.

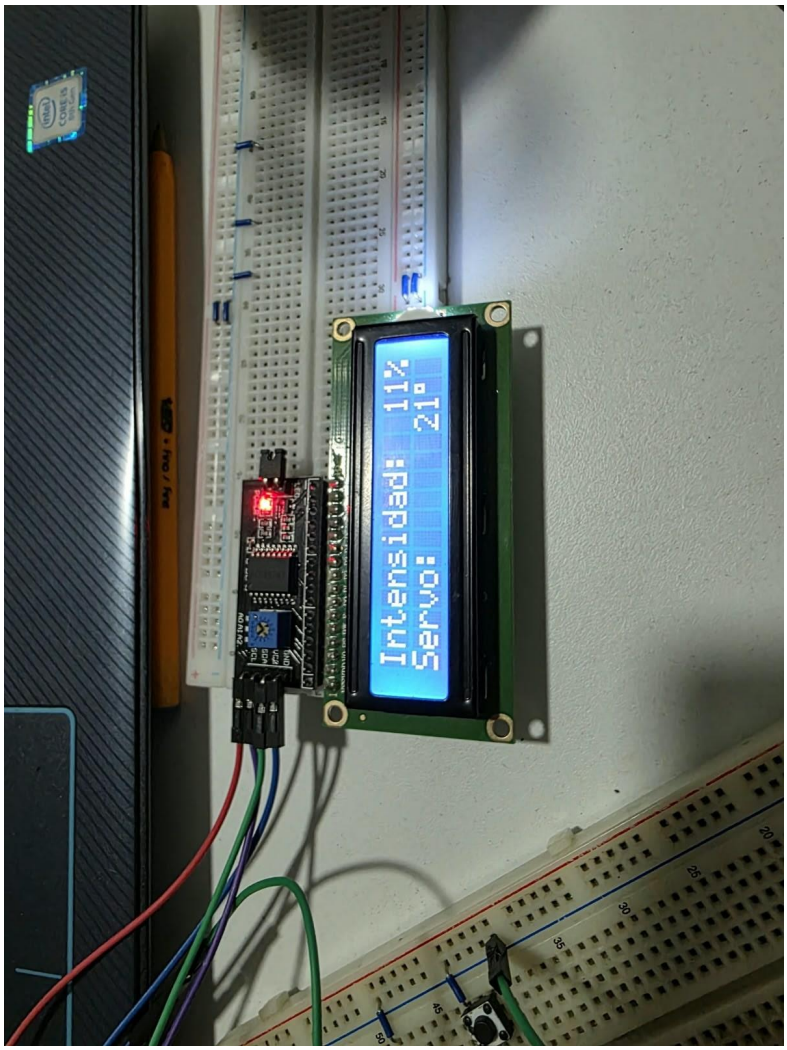


Ilustración 2. Pantalla LCD mostrando una intensidad de 11 % con el servomotor a 21°

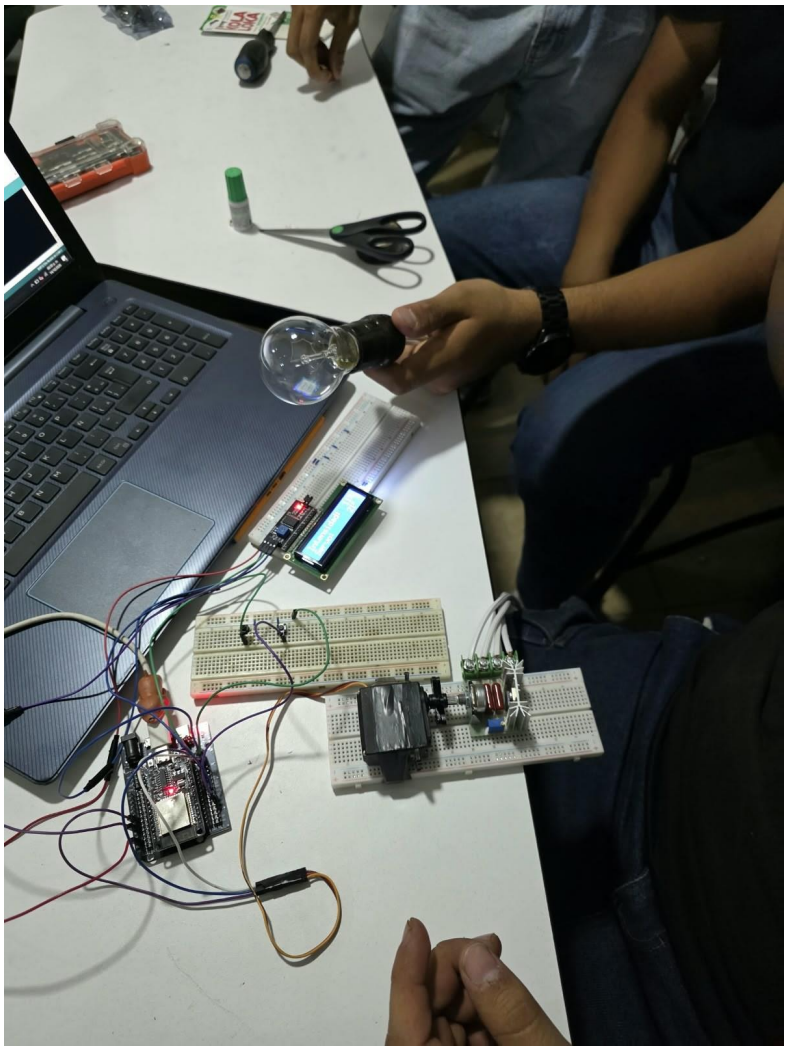


Ilustración 3. Circuito ensamblado con el foco a baja intensidad

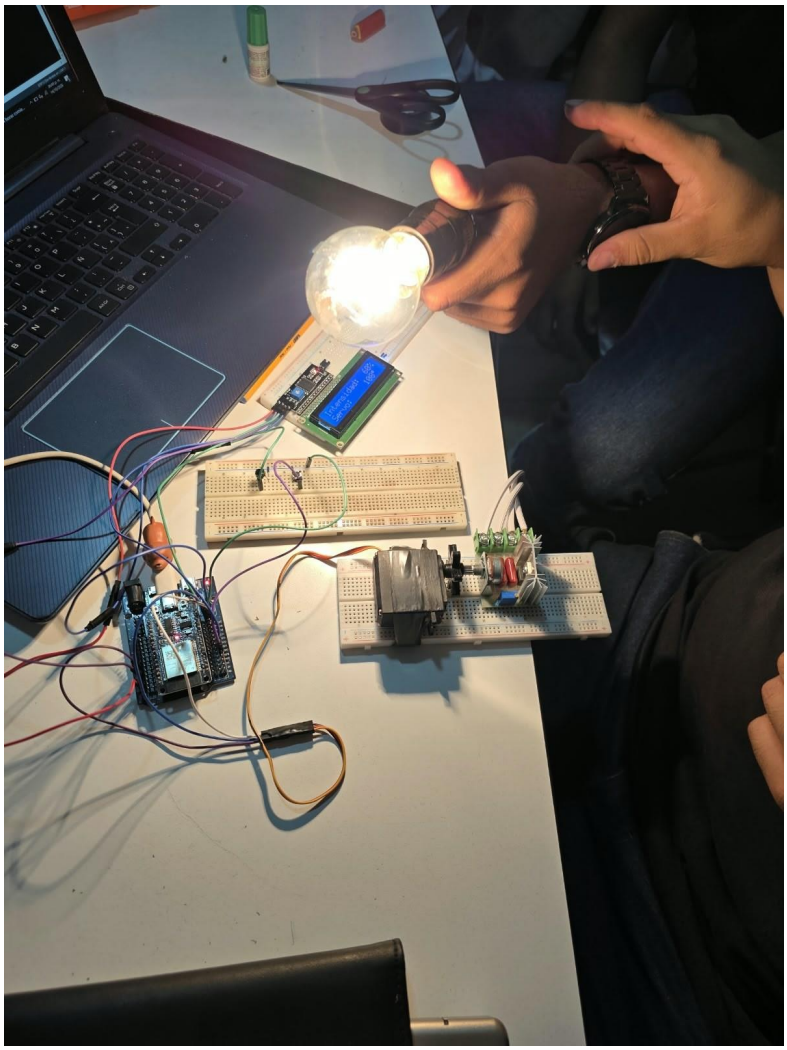


Ilustración 4. Foco encendido con una intensidad de 60 % (servomotor a 108°)

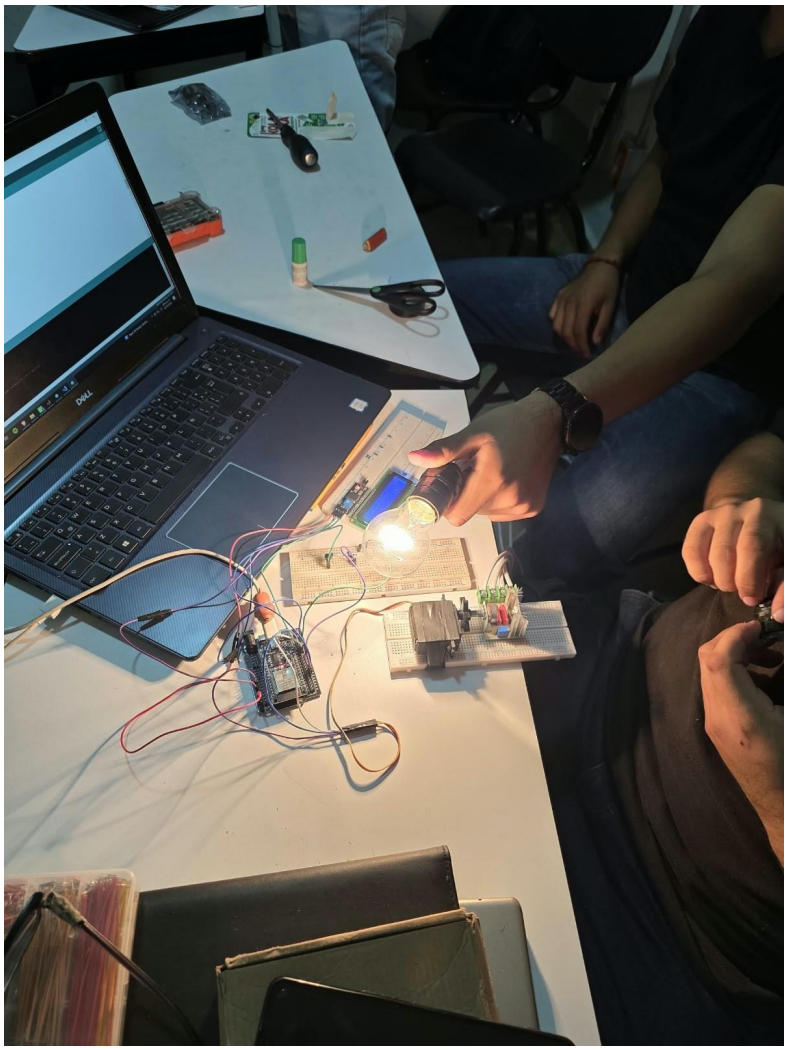


Ilustración 5. Vista general del circuito en funcionamiento

Observaciones y Conclusiones

Fue una práctica sencilla, con pocas conexiones, que no requirió mucho tiempo para realizarse. El acople entre las estrellas del servo y el potenciómetro funcionó correctamente, aunque fue necesario reforzarlo con cinta para que no se moviera. Alimentar el servomotor desde la salida de 5 V de la ESP32 fue suficiente para mover la perilla del dimmer.

Durante las pruebas, a uno de los integrantes del equipo le dio un toque al tomar el dimmer por la parte de abajo de su placa, ya que las pistas de ese lado están expuestas y conectadas a 120 V. Por ello, el módulo solo debe manipularse cuando está desconectado.

En conclusión, se logró controlar la intensidad de un foco mediante un dimmer AC accionado por un servomotor y una ESP32, utilizando dos botones para ajustar la posición y una pantalla LCD para visualizar el porcentaje de intensidad y los grados del servo.

Referencias

[1] Espressif Systems. (s. f.). ESP32 Series Datasheet. Espressif Systems.

[2] Arduino. (s. f.). Arduino Documentation. Arduino.

[3] Tower Pro. (s. f.). *MG995 Metal Gear Servo* [Hoja de datos]. Tower Pro.

[4] Harrington, K. y Bennett, J. K. (s. f.). *ESP32Servo* [Librería de Arduino]. GitHub.